

Gateway SSW: de curl para fetch

Transporte HTTP nativo, timeout, retry e charset explícito

Comparação: commit f7d49ae (HEAD) x working tree
Branch: v2-rastreamento
Data: 29 de julho de 2026
Arquivos principais: src/services/ssw/tracking/getTracking.gateway.ts

Resultado: o runtime deixa de criar subprocessos e passa a consultar o SSW com APIs nativas do Node.

Antes

- `execFile('curl', args)` e `promisify`.
- Dependência do binário `curl` e do ambiente filtrado por `getRuntimeEnv`.
- `--max-time`, `-L`, headers e body montados como argumentos.
- Retry único quando o exit code era 28.
- `maxBuffer` de 2 MB.

Agora

```
fetchSsw(url, options, attempt)
-> AbortController
-> fetch
-> response.ok
-> arrayBuffer
-> TextDecoder('iso-8859-1')
```

Aspecto	Antes	Agora
Transporte	subprocesso curl	fetch nativo
Timeout	<code>curl --max-time</code>	<code>AbortController</code>
Retry	exit code 28	<code>AbortError/TimeoutError</code>
Charset	string padrão	ISO-8859-1 explícito
4xx/5xx	HTML seguia para validação	<code>SSW_UNAVAILABLE</code> imediato
Tamanho	limite de 2 MB	sem limite próprio

Ganhos

- Maior compatibilidade com ambientes serverless.
- Menor overhead por evitar processos externos.
- Headers comuns centralizados.
- Decodificação mais adequada ao HTML histórico do SSW.
- Erros HTTP não bem-sucedidos identificados imediatamente.

Riscos técnicos

- O timer é cancelado antes de `response.arrayBuffer()`, então pode não limitar o body completo.
- A resposta inteira é carregada em memória sem limite explícito.
- O fetch não declara `cache: 'no-store'`.
- `form.method` é capturado, mas o envio continua fixo em POST.
- A dependência circular permanece: gateway importa core e core importa gateway.

Etapa anterior reaproveitada do relatório Markdown

Antes da troca de transporte, a refatoração já havia removido `src/utils/scrapers/tracking.scaper.ts` e `src/utils/parsers/tracking.parser.ts`. O scraping foi incorporado ao gateway SSW e o parser foi recriado ao lado dele. A mudança atual mantém essa organização e substitui somente a execução de curl pelo fetch nativo.

Etapa	Organização	Transporte
Antes da refatoração	service + scraper/parser em <code>src/utils</code>	curl
Após a refatoração	core + gateway/parser em <code>services/ssw</code>	curl
Estado atual	mesma separação, com caches no core	fetch nativo